ServoControlWpf 설계 과정과 운용 원리

Windows(WPF) 에서 WSL 과 Raspberry Pi 4 B 를 거쳐 HiTEC HS-311 서보를 움직이는 프로그램 — 왜, 무엇을, 어떻게 만들었나

작성 정재현 (Claude Code 와 함께 작성) · 2026. 10. 26. · Rev. 1.0 · 함께 보는 문서: *ServoControl 동작 원리와 소스코드*, *Raspberry Pi 4B + HS-311 데이터시트*

> **이 문서는 누구를 위한 것인가**
>
> 처음 보는 사람도 따라올 수 있게 썼습니다. 각 장은 **왜 → 무엇을 → 어떻게** 순서입니다. 프로그램을 **쓰는 방법**만 알고 싶으면 10장 "운용 원리와 기법" 과 13장 "만들기·실행하기" 를 먼저 보세요.

차례

1. 한눈에 보기
2. 출발점 — 기존 Qt 프로그램이 하던 일
3. 요구사항 정리와 결정 과정
4. 세 가지 연결 길 — 직접 · WSL 터널 · 에뮬레이터
5. 프로그램 구조 — Qt MVC → WPF MVVM
6. 데이터 흐름 — 다이얼을 돌리면 일어나는 일
7. pigpiod 16바이트 명령
8. WSL 접근 — WPF 가 리눅스를 부르는 법
9. 화면 설계
10. 운용 원리와 기법
11. 안전 장치 (소프트웨어)
12. 테스트와 검증 결과
13. 만들기·실행하기
14. 한계와 다음 단계

1. 한눈에 보기

한 줄 요약: 서보는 "펄스 길이" 로 각도를 받는다. 그 펄스를 Raspberry Pi 의 **pigpiod** 가 만들어 주고, Windows 의 WPF 프로그램은 "GPIO18 을 1500 µs 로" 같은 **16바이트 명령**을 TCP 로 보내기만 한다. 명령을 보내는 길은 세 가지 — **직접**, **WSL 터널**, **에뮬레이터**(가짜 Pi) — 중에서 고른다.

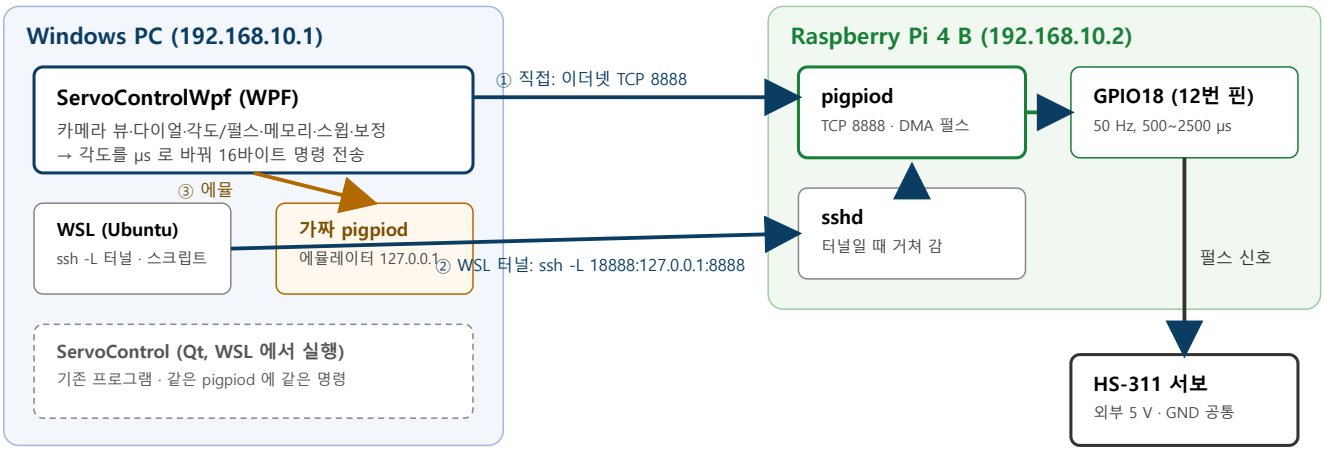


그림 1. 전체 구성. 서보를 직접 움직이는 것은 언제나 Pi 의 pigpiod 다. WPF 는 "어느 길로 명령을 보내느냐" 만 다르다.

연결 방식	명령이 가는 길	언제 쓰나
① **직접 연결**	WPF → 이더넷 → Pi `192.168.10.2:8888`	평소. Pi 에 `pi_setup.sh` 로 원격 허용을 해 둔 경우. 가장 빠름
② **WSL 터널**	WPF → `wsl.exe` → `ssh -L` → Pi 의 `127.0.0.1:8888`	통신을 WSL/SSH 로 보내고 싶을 때. Pi 쪽 원격 허용 설정이 없어도 됨
③ **에뮬레이터**	WPF → `127.0.0.1` → 앱 안의 가짜 pigpiod	Pi 가 없을 때. 실제와 같은 TCP 명령 경로를 그대로 검증

2. 출발점 — 기존 Qt 프로그램이 하던 일

새 프로그램을 만들기 전에, 먼저 있던 것을 정확히 알아야 합니다. 폴더 `ServoChapter\ServoControl` 에 Qt/C++ 프로그램과 PDF 3종이 있었습니다.

2-1. 서보는 어떻게 움직이나 (쉬운 설명)

RC 서보는 **"얼마나 긴 전기 신호를 받았느냐"** 로 자기 팔(혼)을 어디에 둘지 정합니다. 20 ms(1초에 50번) 마다 짧은 High 펄스가 오는데, 그 펄스의 **길이**가 각도입니다.

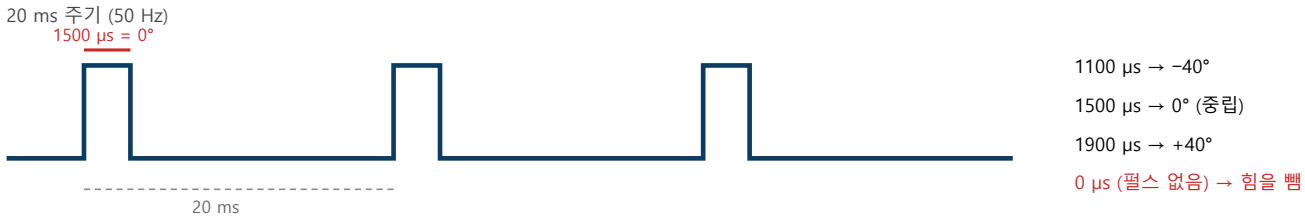


그림 2. HS-311 은 중립 1500 µs 에서 10 µs 마다 1° 씩 움직인다 (제조사 공식 범위 ±40° = 1100~1900 µs).

> **비유**
> 서보는 "같은 말을 계속 들어야 그 자리를 지키는" 장치입니다. 펄스를 끊으면(0) 힘을 빼고 손으로 돌릴 수 있게 됩니다. 그래서 프로그램에는 항상 **[출력 끄기]** 버튼이 있고, 종료할 때도 꼭 0 을 보냅니다.

2-2. 기존 구조 (Qt MVC)

Qt 파일	계층	역할
`model/servospec.h`	Model	HS-311 사양, 각도↔펄스 변환, 값 검사
`model/servodriver.h`	Model	"펄스를 내보낸다" 는 약속(인터페이스). 원격/시뮬레이션 교체 가능
`model/remotepigpiodriver.*`	Model	QTcpSocket 으로 pigpiod 에 16바이트 명령 전송
`model/servomodel.*`	Model	현재 각도·출력 상태, 드라이버 교체, `changed()` 신호
`view/servogauge.*`, `view/mainwindow.*`	View	반원 게이지(드래그), 다이얼, 버튼, 위치 메모리, 스윕, 보정 화면
`controller/servocontroller.*`	Controller	요청 처리, 연결/해제, 스윕 타이머, 설정 저장, 오류 처리
`pi_setup.sh · deploy.sh · pi_info.sh`	스크립트	pigpio 설치·원격 허용 / Pi 로 전송·빌드·실행 / Pi 정보 수집 (WSL 에서 실행)

핵심 사실 세 가지를 PDF 와 코드에서 확인했습니다.

- **통신 규약**: TCP 8888, 요청·응답 모두 16바이트, 명령은 SERVO(8)·HWVER(17)·PIGPV(26) 세 개뿐.
- **네트워크**: PC 192.168.10.1 ↔ Pi 192.168.10.2 이더넷 1:1. SSH 별명 `rpi`, 키 인증. pigpiod 는 127.0.0.1 과 192.168.10.1 만 허용.
- **안전 규칙**: 범위 밖 각도는 잘라서 출력, 통신 끊기면 자동 해제 후 재접속해 0 전송, 종료 시 펄스 0.

3. 요구사항 정리와 결정 과정

요청은 **"WPF 도 WSL 에 접근해서 라즈베리파이를 제어하는 프로그램"** 이었습니다. 이 한 문장에는 해석이 여러 가지라서, 만들기 전에 네 가지를 물어 결정했습니다.

질문	선택지	결정	이유
WPF 가 Pi 로 명령을 보내는 길은?	(a) 직접 TCP + WSL 은 스크립트용 (b) WSL SSH 터널만 (c) 둘 다 선택 가능	**(c) 둘 다**	직접 연결은 빠르고 간단, 터널은 "WSL 을 거친다" 는 요구를 글자 그대로 만족하고 Pi 설정이 없어도 됨. 화면에서 고르게 함
기능 범위는?	Qt 와 동일한 전체 기능 / 핵심 제어만	**전체 기능**	다이얼·위치 메모리·스윕·보정·설정 저장·단축키까지 1:1 이식. 교육용으로 Qt 코드와 대응 관계를 유지. (이후 왼쪽 게이지는 카메라 뷰로, 펄스폭은 입력 가능하게 발전 — 9장)
이 PC 에 WSL 이 없는데?	다른 PC 에서 실행 / 여기 설치 / ssh.exe 대체	**다른 PC 에서 실행**	여기서는 빌드·시뮬레이션·에뮬레이터로 검증. WSL 이 없으면 앱이 안내 메시지를 보여 주도록 함
프로젝트 위치 ·.NET 버전?	net8.0 / net10.0	**`ServoChapter\ServoControlWpf`, net8.0-windows**	LTS. `RollForward=LatestMajor` 로 런타임 10 에서도 실행

작업 중에 추가된 요청 세 가지도 반영했습니다.

추가 요청	반영
설계 과정을 PDF 로, 별도 폴더에, 이해하기 쉽게	이 문서. 폴더 `ServoControlWpf_설계문서`. 기존 PDF 와 같은 방식(HTML → Chromium 인쇄)
프로그램 운용 원리와 기법도 작성	10장 "운용 원리와 기법"
지금 하드웨어가 없어서 에뮬레이션 필요	가짜 pigpiod 를 만들어 ① 앱 안의 "에뮬레이터" 연결 모드, ② 콘솔 프로그램 `pigpio-emu`, ③ 자동 테스트에 사용

> **결정의 공통 원칙**
> **"Pi 쪽은 건드리지 않는다."** pigpiod 와 스크립트는 그대로 두고, Windows 쪽에서만 길을 추가했습니다. 그래서 Qt 프로그램과 WPF 프로그램이 같은 Pi 에 번갈아 접속할 수 있습니다.

4. 세 가지 연결 길 — 직접 · WSL 터널 · 에뮬레이터

4-1. ① 직접 연결 — 왜 Pi 설정을 바꾸지 않아도 되나

Qt 프로그램은 WSL 안에서 돌았는데, WSL2 는 기본적으로 **NAT** 방식이라 WSL 이 보낸 패킷은 Windows 의 주소(192.168.10.1)로 바뀌어 나갑니다. 즉 Pi 의 pigpiod 는 지금까지도 "192.168.10.1 에서 온 접속" 을 받고 있었습니다. Windows 에서 직접 접속해도 Pi 가 보기에는 똑같은 주소이므로, `pi_setup.sh` 가 허용해 둔 목록을 그대로 씁니다.

4-2. ② WSL 터널 — "WSL 을 거쳐서" 를 글자 그대로



그림 3. WSL 터널. WPF 는 자기 PC 의 127.0.0.1:18888 에 접속할 뿐인데, ssh 가 그 데이터를 Pi 의 8888 로 옮겨 준다.

앱이 실행하는 명령은 이것입니다.

```
wsl.exe [-d 배포판] -e ssh -N -o BatchMode=yes -o ExitOnForwardFailure=yes -o ConnectTimeout=5 \
        -L 127.0.0.1:18888:127.0.0.1:8888 rpi
```

옵션	뜻 (쉽게)
`-N`	원격에서 명령을 실행하지 않고 "통로" 만 연다
`-L 127.0.0.1:18888:127.0.0.1:8888`	내 쪽 18888 로 들어온 것을 저쪽(Pi) 의 8888 로 보낸다
`-o BatchMode=yes`	비밀번호나 "yes/no" 질문을 하지 않는다 (창이 없으니 물어봐도 답할 수 없음 → 키 인증 필수)
`-o ExitOnForwardFailure=yes`	포트를 못 열면 바로 종료 → 앱이 실패를 빨리 알 수 있음

앱은 터널을 띄운 뒤 **127.0.0.1:18888 이 열릴 때까지 최대 10초** 기다리고, 그 다음부터는 ①과 똑같은 `RemotePigpioDriver` 로 접속합니다. 이미 누가 18888 을 열어 두었다면(앱이 비정상 종료돼 ssh 가 남은 경우) 새로 띄우지 않고 그대로 씁니다. 연결 해제·종료 시 `Process.Kill(entireProcessTree)` 로 wsl.exe 와 그 안의 ssh 를 함께 닫습니다.

4-3. ③ 에뮬레이터 — 하드웨어 없이 "진짜 경로" 를 시험

Pi 가 없을 때 그냥 화면만 움직이는 "시뮬레이션" 은 통신 코드를 전혀 검사하지 못합니다. 그래서 **진짜 pigpiod 처럼 대답하는 가짜 서버**를 만들었습니다.

- 같은 16바이트 규약으로 응답. SERVO 는 펄스 범위(0, 500~2500)와 GPIO 번호를 검사해 진짜처럼 −7, −2 오류를 돌려줌.
- HWVER → `0xc03111` (Pi 4B), PIGPV → 79 를 돌려주므로 연결 바에 "Pi 리비전 c03111 · pigpio v79" 가 뜸.
- 서보의 **물리 움직임**도 흉내: 목표 각도를 향해 0.19 s/60°(HS-311 @4.8 V) 속도로 움직이는 "현재 각도" 를 계산.

- 앱은 에뮬레이터 모드에서도 ①과 **완전히 같은** `RemotePigpioDriver` 를 쓴다. 다른 점은 접속 주소가 127.0.0.1 이라는 것뿐.

> **에뮬레이터로 확인된 것**
> 상단 바에 "가짜 Pi 수신 #138: SERVO gpio=18 출력 끔" 처럼 **가짜 Pi 가 실제로 받은 바이트**가 표시됩니다. 즉 "화면이 움직였다" 가 아니라 "TCP 로 올바른 명령이 도착했다" 를 눈으로 확인합니다.

4-4. 비교

	① 직접	② WSL 터널	③ 에뮬레이터
필요한 것	Pi + pigpiod 원격 허용 (pi_setup.sh)	Pi + WSL + ssh 키 인증(별명 rpi)	없음
명령 지연	LAN 수 ms	SSH 암호화 추가, 보통 10 ms 이내	1 ms 이내
Pi 가 보는 접속 주소	192.168.10.1	127.0.0.1	(해당 없음)
장점	가장 단순·빠름	암호화, Pi 설정 불필요, "WSL 경유" 요구 충족	Pi 없이 통신 코드 검증, 교육·시연
단점	pigpiod 가 허용 주소를 알아야 함	WSL·ssh 준비 필요, 터널 여는 데 1~3초	실제 서보 반응은 볼 수 없음

5. 프로그램 구조 — Qt MVC → WPF MVVM

Qt 의 **MVC**(Model-View-Controller) 를 WPF 의 **MVVM**(Model-View-ViewModel) 으로 옮겼습니다. 이름만 다르고 역할은 거의 같습니다. 가장 큰 차이는 View 와 ViewModel 이 **"바인딩"** 으로 자동 연결된다는 점입니다 — Qt 의 `connect(signal, slot)` 을 XAML 한 줄이 대신합니다.

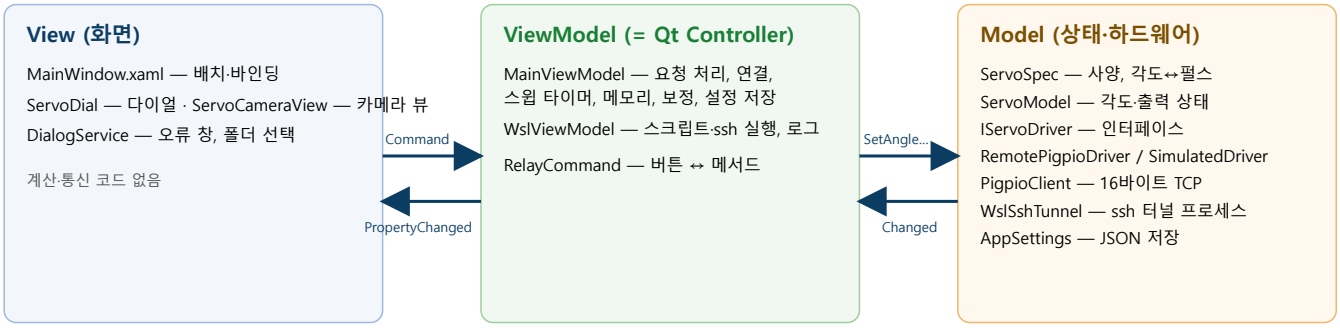


그림 4. 세 계층. View 는 "명령" 만 보내고, ViewModel 이 Model 을 바꾸고, Model 이 Changed 로 알리면 ViewModel 이 PropertyChanged 로 화면을 갱신한다.

Qt (C++)	WPF (C#)	비고
`model/servospec.h`	`Model\ServoSpec.cs`	`record struct`. 변환식 그대로. 테스트 11개
`model/servodriver.h`	`Model\IServoDriver.cs`	실패는 `bool + QString*` 대신 `ServoException`(`LinkLost` 플래그) 으로
`model/remotepigpiodriver.*`	`Model\PigpioClient.cs` + `RemotePigpioDriver.cs`	바이트 처리(PigpioClient) 와 서보 동작(Driver) 을 둘로 나눠 가짜 서버로 단위 테스트 가능
(없음)	`Model\WslSshTunnel.cs`	신규. 터널 프로세스의 수명 관리
`model/servomodel.*`	`Model\ServoModel.cs`	`changed()` → `Changed` 이벤트
`QSettings` (.conf)	`Model\AppSettings.cs` (JSON)	`%APPDATA%\VI\ServoControl.json`
`controller/servocontroller.*`	`ViewModel\MainViewModel.cs`	슬롯 하나 = 메서드 하나. `QSignalBlocker` → `_syncing` 플래그
(없음)	`ViewModel\WslViewModel.cs`, `Services\WslService.cs`	신규. wsl.exe 호출, 스크립트/ssh 실행, 로그
`view/servogauge.*` (반원 게이지, 드래그 입력)	`View\ServoCameraView.cs`	역할이 바뀜: **서보에 올려진 카메라의 움직임**을 위에서 본 그림으로 보여 주는 **표시 전용** 화면. 마우스 입력 없음, 서보 속도로 회전 애니메이션
`QDial` (Qt 기본 위젯)	`View\ServoDial.cs`	WPF 에 다이얼이 없어 직접 만든 원형 노브. 270° 스윕, 눈금, 드래그·휠·방향키. UI 자동화 RangeValue 패턴 지원
`view/mainwindow.*`	`View\MainWindow.xaml(.cs)`	배치는 XAML, 코드 비하인드는 전체화면·Enter·휠·스크롤 네 가지만
`main.cpp`	`App.xaml.cs`	조립, 종료 시 펄스 0
(없음)	`PigpioEmulator\`, `PigpioEmulatorCli\`	신규. 가짜 pigpiod 라이브러리 + 콘솔

5-1. 폴더 구조

```
ServoChapter\
├─ ServoControl\                기존 Qt (수정 안 함; 스크립트 3개를 WPF 가 재사용)
├─ ServoControlWpf\             ★ WPF 솔루션
│  ├─ ServoControlWpf.sln · README.md
│  ├─ ServoControlWpf\          앱  (Model · ViewModel · View · Services)
│  ├─ ServoControlWpf.Tests\    xUnit 34개
│  ├─ PigpioEmulator\           가짜 pigpiod 라이브러리
│  └─ PigpioEmulatorCli\        가짜 pigpiod 콘솔 (pigpio-emu)
└─ ServoControlWpf_설계문서\     ★ 이 문서 (HTML 원본 + PDF + 화면 캡처)
```

6. 데이터 흐름 — 슬라이더를 움직이면 일어나는 일

Qt 문서의 "다이얼을 돌렸을 때" 와 같은 7단계입니다. 번호를 따라가 보세요.

1. **MainWindow**: 슬라이더 값이 바뀜 → 바인딩이 `MainViewModel.Angle` 에 값을 넣음
2. **MainViewModel.Angle setter**: 스윕 중이면 멈추고 `_model.SetAngle(angle)`
3. **ServoModel.SetAngle**: 범위로 자른 뒤 `ServoSpec.AngleToPulse()` 로 펄스 계산 → `driver.WritePulse(µs)`
4. **RemotePigpioDriver → PigpioClient**: SERVO(8, 18, µs) 16바이트 전송 → 응답 `res` 확인
5. **pigpiod** (또는 가짜 pigpiod): GPIO18 펄스 폭 변경 → 서보 이동
6. **ServoModel**: 성공하면 각도 저장 후 `Changed` 이벤트
7. **MainViewModel.SyncFromModel**: `_syncing = true` 상태로 Angle·Pulse·IsOutputOn 등 `PropertyChanged` → 다이얼·각도 줄·펄스 줄 갱신, 왼쪽 카메라 뷰는 (연결된 상태라면) 새 목표를 향해 서보 속도로 회전 시작. 연결 해제 상태면 카메라 뷰는 움직이지 않음

> **무한 루프 막기**
> 7번에서 슬라이더 값을 다시 넣을 때, 슬라이더는 또 "값이 바뀌었다" 고 ViewModel 에 알립니다. 그냥 두면 "갱신 → 요청 → 갱신 → …" 이 반복됩니다. Qt 는 `QSignalBlocker` 로 막았고, WPF 판은 `_syncing` 플래그가 켜져 있으면 `Angle` setter 가 값만 저장하고 명령을 보내지 않습니다.

6-1. 펄스폭으로 조작하면

펄스 줄(슬라이더·입력칸)을 바꾸면 ViewModel 의 `PulseInput` setter 가 받습니다. 사양 범위(최소~최대 펄스)로 자른 뒤 `ServoSpec.PulseToAngle()` 로 각도를 구해 위 2번부터 같은 길을 갑니다. 즉 **모델이 기억하는 것은 언제나 "각도" 하나**이고, 펄스폭은 사양으로 계산한 값입니다. 그래서 7번 갱신 때 각도 줄·다이얼·카메라 뷰·펄스 줄이 모두 같은 상태를 보여 주고, 보정값을 바꾸면 펄스 줄의 범위와 값도 함께 바뀝니다.

입력	따라 바뀌는 것
각도 줄 / 다이얼 / ± 버튼	펄스 줄 = `AngleToPulse(각도)`, 서보 출력, 카메라 뷰 회전
펄스 줄 (예 1650 µs)	각도 = `PulseToAngle(1650)` = 15.0°, 다이얼·각도 줄, 서보 출력, 카메라 뷰 회전
범위 밖 펄스 (예 2400, 공식값일 때)	최대 1900 µs 로 잘려 40°
보정 적용 (예 확장값)	펄스 줄 범위 600~2400, 현재 각도에 맞는 펄스로 다시 계산

6-2. 실패했을 때

4번에서 실패하면 `ServoException` 이 올라옵니다. `LinkLost` 가 true(연결 끊김·시간 초과) 이면 ViewModel 이 **먼저 연결을 정리**(시뮬레이션 드라이버로 교체, 이전 드라이버가 재접속해 0 전송) 하고 오류 창을 띄웁니다. pigpio 가 거부한 오류(-7 등) 는 연결을 유지한 채 메시지만 보여 줍니다. 어느 경우든 Model 의 각도는 바뀌지 않으므로 화면이 "실제 서보 상태" 와 어긋나지 않습니다.

7. pigpiod 16바이트 명령

pigpiod 는 TCP 8888 로 명령을 받습니다. 요청과 응답은 모두 **4바이트 정수 4개**(리틀엔디언 = 낮은 바이트가 먼저) 입니다.

구분	바이트 0~3	4~7	8~11	12~15
요청	cmd (명령 번호)	p1	p2	p3 (= 0)
응답	cmd (같은 값)	p1	p2	res (0 이상 성공, 음수 오류)

명령	번호	p1 / p2	용도
SERVO	8	GPIO 번호 / 펄스폭(µs)	서보 펄스 설정. 0 이면 출력 끔. 500~2500 만 허용
HWVER	17	— / —	연결 직후 Pi 하드웨어 리비전 읽기 (예 0xc03111 = Pi 4B)
PIGPV	26	— / —	연결 직후 pigpio 버전 읽기

```
예) "GPIO18 을 1500 µs 로"   →  08 00 00 00 · 12 00 00 00 · dc 05 00 00 · 00 00 00 00
    (8 = SERVO, 18 = 0x12, 1500 = 0x05dc)
C# 에서는  BinaryPrimitives.WriteUInt32LittleEndian(req[0..4], cmd)  처럼 넣는다.
```

이 바이트 열이 정확히 나가는지는 자동 테스트 `Servo_SendsExactly16BytesLittleEndian` 이 가짜 서버로 받아서 확인합니다. 오류 코드는 한글로 바꿔 보여 줍니다: −2 "사용할 수 없는 GPIO 번호", −7 "펄스폭은 0 또는 500~2500 µs", −41 "권한 없음".

8. WSL 접근 — WPF 가 리눅스를 부르는 법

Windows 프로그램이 WSL 을 쓰는 가장 확실한 방법은 `wsl.exe` 를 실행하는 것입니다. WPF 앱은 다음 네 가지 모양으로 부릅니다.

하는 일	실행하는 명령	비고
WSL 이 있나, 배포판은?	`wsl.exe -l -q`	출력이 **UTF-16LE** → `Encoding.Unicode` 로 읽어야 함 (안 그러면 글자 사이에 빈 문자가 끼어 깨짐)
스크립트 실행	`wsl.exe [-d 배포판] --cd "D:\…\ServoControl" -e bash ./pi_info.sh rpi`	`--cd` 가 Windows 경로를 `/mnt/d/…` 로 자동 변환. 출력은 UTF-8
Pi 에 명령 한 줄	`wsl.exe -e ssh -o BatchMode=yes -o ConnectTimeout=5 rpi "systemctl is-active pigpiod"`	키 인증 필수 (비밀번호를 받을 창이 없음)
터널 열기	`wsl.exe -e ssh -N -L 127.0.0.1:18888:127.0.0.1:8888 rpi`	4-2 절

왜 pi_setup.sh 만 새 터미널 창에서 여나

`pi_setup.sh` 는 Pi 에서 `sudo apt-get install` 을 하고 `ssh -t` 로 접속합니다. 비밀번호를 물어볼 수 있는데, 출력을 가로채는 "보이지 않는 실행" 에서는 답을 입력할 방법이 없어 멈춰 버립니다. 그래서 이 스크립트만 **콘솔 창을 그대로 띄워**(`CreateNoWindow=false`) 사람이 직접 입력할 수 있게 했습니다. 나머지(`pi_info.sh`, `deploy.sh`, ssh 명령) 는 출력을 앱의 로그 창에 실시간으로 보여 줍니다.

모든 외부 실행은 `Services\ProcessRunner` 한 곳을 거칩니다: 줄 단위로 stdout/stderr 를 받아 UI 스레드로 넘기고, [취소] 를 누르면 프로세스 트리를 통째로 끝냅니다. WSL 이 없는 PC 에서는 "이 PC에 WSL이 설치되어 있지 않습니다. 'wsl --install' 로 설치하세요." 를 보여 주고 앱은 계속 동작합니다(이 PC 에서 확인됨).

9. 화면 설계

Qt 화면과 같은 세 칸 배치입니다. 오른쪽 칸은 탭 두 개("서보", "WSL / Pi 관리") 로 나눴습니다.

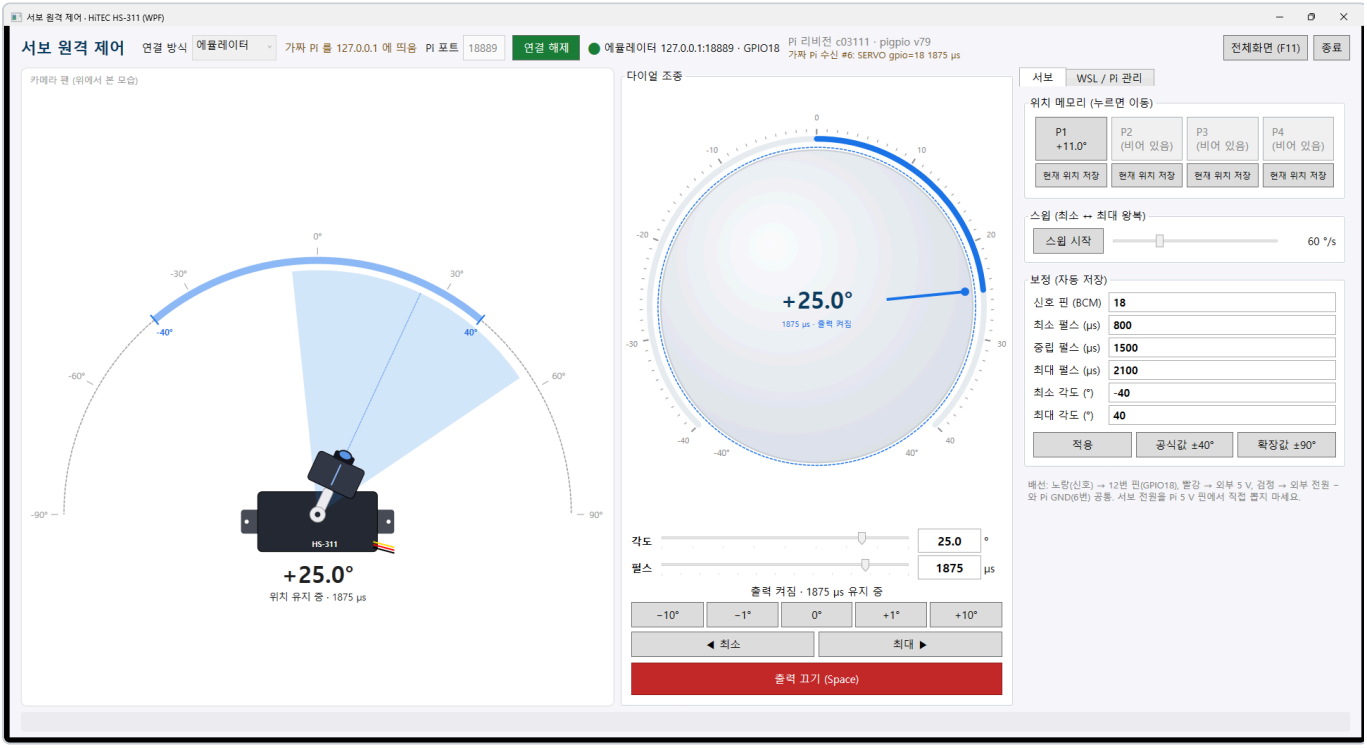


그림 5. 에뮬레이터에 연결해 다이얼로 +25° 를 준 화면. 왼쪽은 서보 위 카메라가 +25° 를 향해 시야(파란 쐐기)를 비추는 모습, 가운데는 원형 다이얼과 그 아래 각도 줄(25.0°)·펄스 줄(1875 µs)이 서로 묶여 있다(보정값 800/1500/2100 µs 기준). 상단에 "Pi 리비전 c03111 · pigpio v79" 와 가짜 Pi 가 받은 SERVO 명령(1875 µs)이 보인다.

영역	위젯	내보내는 명령 (ViewModel)
상단 연결 바	연결 방식 콤보(직접/WSL 터널/에뮬레이터), Pi 주소·포트 또는 ssh 별명·배포판·로컬 포트, [연결] 토글, ● 표시등, 장치 정보, 에뮬 수신 표시	`ConnectToggleCommand`
왼쪽 카메라 뷰	`ServoCameraView` — 위에서 내려다본 서보 본체(HS-311)와 혼 위의 카메라, 카메라 시야(FOV 62°) 쐐기, 허용 범위 호와 끝 표시, 30° 눈금, 현재 각도·목표·펄스·상태 글자. **표시 전용**(마우스 입력 없음). **연결됐을 때만** 움직이고, 해제 상태에서는 "연결 안 됨" 안내만 표시	없음 (`Angle` · `IsOutputOn` · `Pulse` · `IsConnected` 를 읽기만 함)
가운데 다이얼 조종	**원형 다이얼** `ServoDial` (노브를 끌어 돌림, 휠 1칸 = 1°, ←/→ 1°, PgUp/PgDn 10°, 가운데 각도·펄스폭 표시), **각도 줄**(슬라이더 + 입력칸 °) 과 **펄스 줄**(슬라이더 + 입력칸 µs) — 어느 쪽을 바꿔도 다른 쪽이 따라 바뀜, 출력 상태, −10/−1/0°/+1/+10, 최소/최대, [출력 끄기]	`Angle` · `PulseInput` 양방향 바인딩, `NudgeCommand`, `CenterCommand`, `GoMin/GoMaxCommand`, `ReleaseCommand`
탭 "서보"	위치 메모리 P1~P4(이동 + 현재 위치 저장), 스윕 시작/정지 + 속도 5~200 °/s, 보정 6칸 + [적용][공식값][확장값], 배선 메모	`PresetRecall/StoreCommand`, `ToggleSweepCommand`, `ApplySpec/Official/ExtendedCommand`
탭 "WSL / Pi 관리"	WSL 상태 + [WSL 확인], 배포판·ssh 별명·스크립트 폴더, [Pi 상태 확인][pi_info.sh][deploy.sh][pi_setup.sh ↗][취소], ssh 명령 입력, 검은 로그 창	`WslViewModel` 의 커맨드들
상태 바	4초 뒤 사라지는 안내문	`StatusText`

키보드: ←/→ = 1° 이동, Space = 출력 끄기, F11 = 전체화면. 모양은 Qt 스타일시트를 옮겼습니다(제목 22px `#0b3d63`, 출력 끄기 빨강 `#c62828`, 연결됨 초록 `#1a7f37`).

9-1. 다이얼은 어떻게 만들었나

WPF 에는 Qt 의 `QDial` 같은 기본 다이얼이 없습니다. 처음에는 슬라이더로 대신했지만 "다이얼 조종" 이라는 이름과 맞지 않아, `FrameworkElement` 를 상속해 **직접 그리는 노브**를 만들었습니다.

- **그리기**: 값 범위(최소~최대)를 화면 각도 −135°~+135°(270°) 에 비례로 펼친다. 바깥에 1° 눈금과 10° 숫자, 안쪽에 0° 부터 현재 값까지 파란 호, 가운데에 노브와 포인터, 노브 안에 각도 숫자.
- **조작**: 노브를 누르거나 끌면 중심에서 본 마우스 방향각을 `atan2` 로 구해 값으로 바꾼다(1° 단위로 맞춤). 아래쪽 빈 90° 구간을 가리키면 가까운 끝으로. 휠 1칸 = 1°, 포커스가 있으면 ←/→ 1°, PgUp/PgDn 10°, Home/End 끝.
- **연결**: `Value` 가 양방향 바인딩이라 `ViewModel.Angle` 과 바로 묶인다. 모델이 바뀌면 다이얼이 따라 돌고, 다이얼을 돌리면 6장의 흐름대로 서보 명령이 나간다.
- **검증**: "마우스 위치 → 값" 계산은 단위 테스트 10개로, 실제 조작은 UI 자동화(RangeValue 패턴·마우스 메시지)로 확인.

9-2. 왼쪽 화면 — 서보에 올려진 카메라의 움직임

처음에는 왼쪽에 반원 "미터기" 게이지를 두고 드래그로도 조종하게 했지만, 조종은 다이얼 하나로 충분하고 왼쪽은 **"지금 모터가 실제로 어떻게 움직이는가"** 를 보여 주는 편이 직관적이라 바꿨습니다. 이 프로젝트의

서보는 카메라를 좌우로 돌리는(팬) 용도이므로, 위에서 내려다본 서보 본체 위에 카메라를 올린 그림을 그립니다.

- **표시 전용**: `IsHitTestVisible = false` 로 마우스를 전혀 받지 않습니다. 눌러도 각도가 바뀌지 않습니다(자동 테스트로 확인).
- **실제 속도로 움직임**: 바인딩된 `Angle` 은 "명령한 목표" 이고, 그림의 카메라는 그 목표를 향해 HS-311 속도(0.19 s/60° ≈ 316 °/s)로 회전합니다. 그래서 −40° → +40° 명령을 주면 약 0.25 초 동안 돌아가는 모습이 보이고, 글자에 "→ 목표 +40.0° 로 회전 중" 이 표시됩니다.
- **시야(FOV)**: 카메라 앞으로 62°(Pi 카메라 모듈 v2 수평 시야) 쐐기를 반투명하게 그려, 카메라가 어디를 보고 있는지 알 수 있습니다.
- **상태 표현**: 출력이 꺼지면 카메라가 회색이 되고 시야가 사라집니다(힘이 빠진 상태). 허용 범위는 연한 파란 호, 범위 끝에 닿아 있으면 "끝단 — '지잉' 소리가 나면 범위를 줄이세요" 경고가 뜹니다.
- **Model 과의 관계**: 애니메이션은 화면 안에서만 일어납니다. Model 의 각도(= 서보에 보낸 명령)는 즉시 바뀌고, 그림만 뒤따라갑니다. 에뮬레이터도 같은 속도로 "현재 각도" 를 흉내 내므로 둘이 거의 일치합니다.
- **연결됐을 때만 움직임**: `IsConnected` 가 바인딩되어 있어, **연결 해제 상태에서는 다이얼·각도·펄스를 아무리 바꿔도 카메라는 제자리**입니다. 그림은 반투명 막으로 흐려지고 "연결 안 됨 — Pi 또는 에뮬레이터에 [연결]하면 카메라 움직임이 여기에 표시됩니다" 안내만 보입니다. 이유는 단순합니다. 연결이 없으면 서보로 명령이 나가지 않으므로 실제 모터도 움직이지 않는데, 화면이 움직이면 "지금 모터가 돌고 있다" 는 거짓 정보가 됩니다. 구동 조건은 `연결됨 AND 출력 켜짐` 하나로 묶었고(`IsDriven`), 연결 직후 출력이 꺼진 상태에서는 회색 카메라가 마지막 위치에 머물다가 첫 명령으로 출력이 켜질 때 목표를 향해 출발합니다. 연결을 끊으면 회전 중이던 카메라도 그 자리에서 멈춥니다(실제 서보도 힘이 빠지면 그 자리에 머묾).

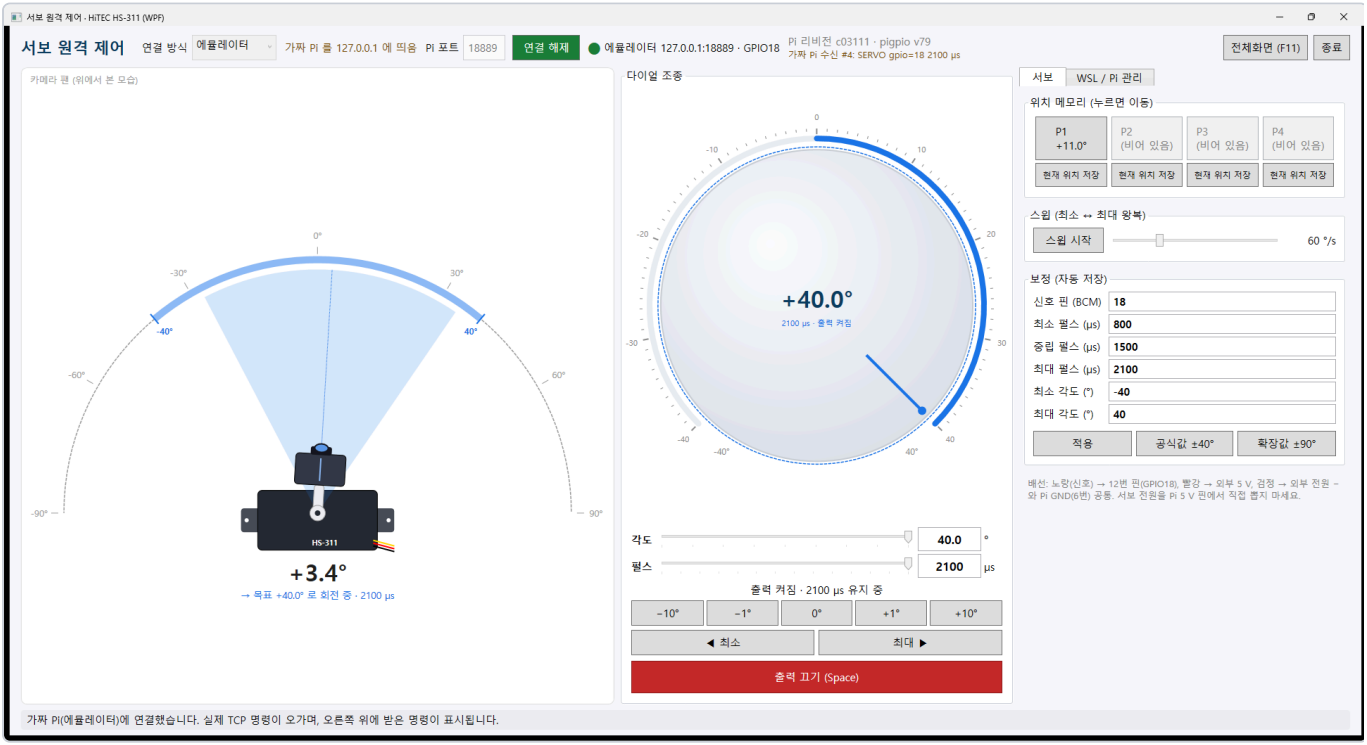


그림 6. −40° 에서 +40° 로 명령한 직후(약 60 ms 뒤) 캡처. 다이얼·펄스 줄은 이미 목표(+40°, 2100 µs)를 가리키지만, 왼쪽 카메라는 +3.4° 를 지나며 "→ 목표 +40.0° 로 회전 중" 이라고 표시한다. 실제 서보가 움직이는 데 걸리는 시간(80° ≈ 0.25 s)을 그대로 보여 준다.

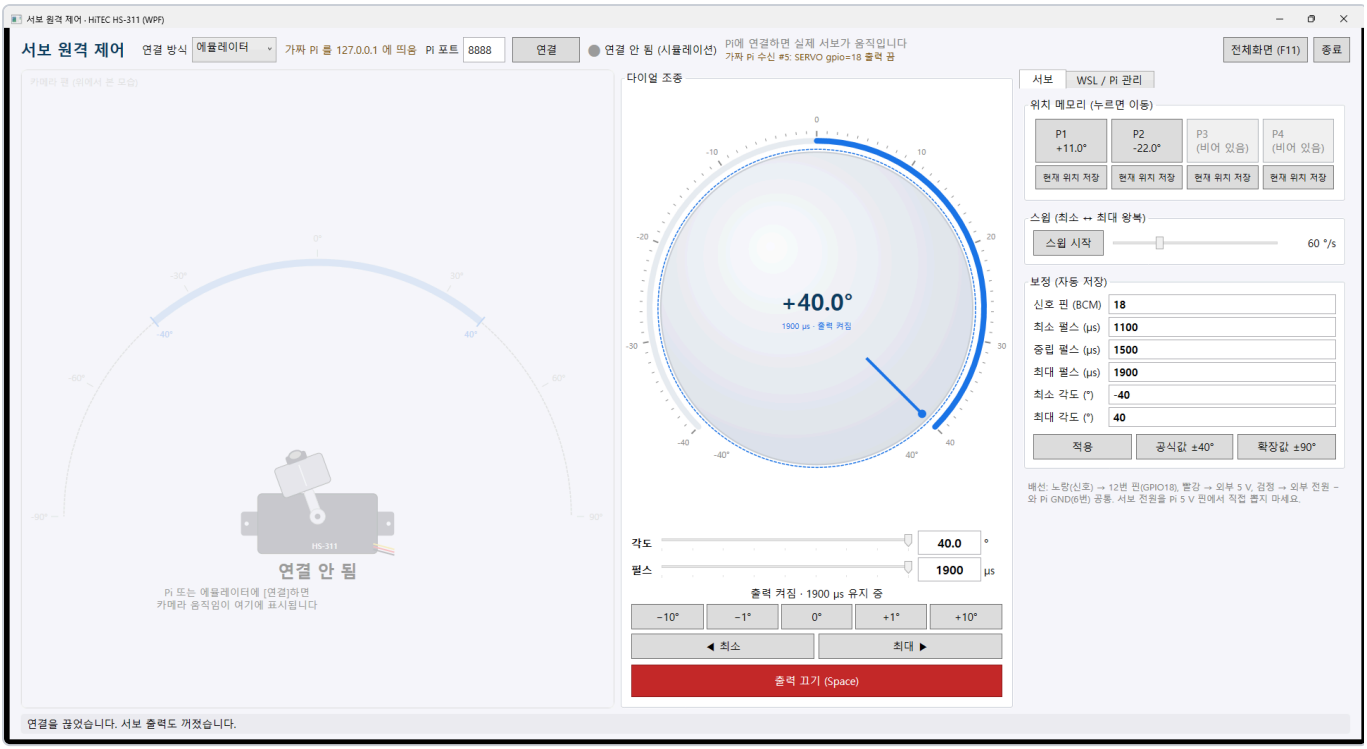


그림 6-1. 연결을 끊은 뒤 다이얼을 +40° 로 돌린 화면. 오른쪽 다이얼·각도 줄·펄스 줄은 +40°, 1900 µs 를 가리키지만, 왼쪽 카메라는 끊기 전 위치(−20°)에 회색으로 멈춰 있고 "연결 안 됨" 안내가 덮여 있다. 명령이 서보로 나가지 않으니 화면도 움직이지 않는다.

10. 운용 원리와 기법

이 장은 "프로그램을 어떻게 돌리고, 왜 그렇게 해야 하는가" 를 다룹니다. 처음 운용하는 사람은 10-6 절차와 10-7 안전 수칙만 먼저 읽어도 됩니다.

10-1. 서보 제어의 원리 — 펄스 폭이 곧 각도

- **프레임**: 20 ms 마다 한 번(50 Hz). 이 주기는 거의 모든 아날로그 서보의 약속입니다.
- **폭**: HS-311 은 1500 µs 가 0°, ±400 µs 가 ±40°. 즉 **10 µs = 1°**. 프로그램의 `AngleToPulse` 가 이 비례식을 계산합니다(중립 기준으로 + 쪽, − 쪽 따로).
- **데드밴드 5 µs**: 펄스가 5 µs 이내로 바뀌면 서보가 움직이지 않습니다. 0.5° 미만 변화는 의미가 없다는 뜻이므로, 프로그램은 1° 단위 조작을 기본으로 둡니다.
- **위치 유지**: 같은 펄스를 계속 받으면 서보는 외부 힘에 저항하며 그 자리를 지킵니다(이때 전류가 흐릅니다). 펄스 0 이면 힘을 뺍니다.
- **속도**: 0.19 s/60°(4.8 V). 80° 를 가려면 약 0.25 초. 스윕 속도를 200 °/s 넘게 올려도 서보가 따라오지 못합니다(최대 약 315 °/s).

10-2. 왜 pigpiod 인가 — "한 번 말하면 계속 해 주는" 데몬

리눅스 프로그램이 GPIO 를 직접 켰다 껐다 하면 운영체제의 다른 작업 때문에 타이밍이 흔들려 서보가 떨립니다. **pigpiod** 는 Pi 의 **DMA 하드웨어**로 펄스를 만들기 때문에 µs 단위로 정확하고, 한 번 "GPIO18 에 1500 µs" 라고 하면 **다음 명령이 올 때까지 스스로 계속** 내보냅니다.

> **운용에 미치는 뜻**
>
> - 프로그램은 **각도가 바뀔 때만** 명령을 보내면 됩니다. 네트워크가 잠깐 느려져도 서보는 마지막 위치를 지킵니다.
> - 반대로 **접속이 끊겨도 펄스는 계속 나갑니다.** 그래서 프로그램은 끊긴 뒤에도 0.5 초 재접속을 시도해 0 을 보내고, 종료 때도 꼭 0 을 보냅니다. 비정상 종료 후 서보가 계속 힘을 주고 있으면 Pi 에서 `pigs s 18 0` 으로 끌 수 있습니다.

10-3. 네트워크 원리 — 누가 누구로 보이나

상황	Pi 의 pigpiod 가 보는 출발 주소	허용되려면
Qt 앱 (WSL2 NAT 안)	192.168.10.1 (Windows 호스트)	`pigpiod -n 127.0.0.1 -n 192.168.10.1` (pi_setup.sh 가 설정)
WPF 직접 연결	192.168.10.1	
WPF WSL 터널	127.0.0.1 (Pi 자신)	기본 설정 그대로 허용
Pi 화면에서 Qt 앱	127.0.0.1	기본 설정 그대로 허용

WSL2 네트워크를 "미러 모드" 로 바꾼 PC 라면 WSL 도 자기 주소를 그대로 쓰므로, 그때는 `pi_setup.sh rpi <WSL 주소>` 로 허용 주소를 하나 더 넣어야 합니다. 터널 모드는 이 문제와 무관합니다.

10-4. SSH 터널 기법 — 준비와 점검

1. WSL 터미널에서 `ssh rpi` 가 **비밀번호 없이** 들어가는지 확인합니다. 안 되면 `ssh-keygen` 후 `ssh-copy-id rpi`.

2. `~/.ssh/config` 에 별명이 있어야 합니다:

```
Host rpi
    HostName 192.168.10.2
    User user
```

3. 손으로 터널을 시험하려면: `ssh -N -L 127.0.0.1:18888:127.0.0.1:8888 rpi` 를 한 창에서 띄우고, Windows 에서 WPF 를 "직접 연결 · 127.0.0.1 · 18888" 로 접속해도 똑같이 됩니다.
4. 앱은 18888 이 이미 열려 있으면 재사용합니다. 남은 터널을 지우려면 WSL 에서 `pkill -f "ssh -N"`.
5. ssh 가 바로 죽으면 앱 오류 창에 ssh 의 stderr(예: "Host key verification failed") 가 그대로 표시됩니다. 그 문장으로 원인을 찾습니다.

10-5. 보정 기법 — 내 서보에 맞추기

서보는 개체마다 중립과 끝단이 조금씩 다릅니다. 보정은 **"중립 → 끝단 → 저장"** 순서입니다.

1. **중립 맞추기**: [0°] 를 누르고 혼이 원하는 중앙을 가리키는지 봅니다. 어긋나면 "중립 펄스" 를 10 µs 씩 바꿔 [적용]. (10 µs = 1°)
2. **끝단 찾기**: 공식값(±40°) 에서 시작해 [최대 ▶] 로 보낸 뒤, 필요하면 "최대 펄스" 를 20 µs 씩 올리며 [적용]. **'지잉' 소리가 계속 나면 기계적 한계에 닿은 것** — 바로 20~30 µs 줄입니다. 최소 쪽도 같은 방법.
3. **각도 표시 맞추기**: 끝단 펄스에 해당하는 실제 각도를 각도기로 재서 "최대 각도/최소 각도" 에 넣으면 화면 각도가 실제와 일치합니다. + 쪽과 − 쪽 기울기가 달라도 프로그램이 따로 계산합니다.
4. **저장**: [적용] 하면 자동 저장됩니다(`%APPDATA%\VI\ServoControl.json`). [공식값 ±40°] 로 언제든 되돌릴 수 있습니다.

> **확장값 ±90° 는 "시험용"**
> 600~2400 µs 는 제조사 보증 범위 밖입니다. 끝단에서 기어가 걸리면 스톨 전류(최대 800 mA)가 흐르고 레진 기어가 상할 수 있습니다. 프로그램이 확인 창을 띄우는 이유입니다.

10-6. 운용 절차 (실제 Pi)

#	할 일	확인할 것
1	배선 확인 후 **서보 외부 5 V 전원 → Pi 전원** 순서로 켬	GND 가 Pi·외부 전원·서보 셋 다 묶였는지. 서보 + 선은 Pi 5 V 핀에 꽂지 않음
2	WPF 실행 → 탭 "WSL / Pi 관리" → [WSL 확인] → [Pi 상태 확인]	로그에 `pigpiod: active`. 아니면 [pi_setup.sh ↗]
3	연결 방식 선택 → [연결]	상단에 "Pi 리비전 … · pigpio v..." 와 초록 ●
4	**[0°]** 먼저	혼이 중앙으로. 안 움직이면 10-9 문제 해결표
5	다이얼, ±1°, ±10°, 각도 줄·펄스 줄로 이동	펄스폭이 10 µs/° 로 따라 바뀌고, 왼쪽 카메라가 그 방향으로 돌아감. 실제 서보와 그림이 같이 움직이는지 확인
6	필요하면 10-5 보정, 위치 메모리 P1~P4 저장	
7	스윕은 낮은 속도(30~60 °/s) 부터	끝단에서 소리가 나면 범위를 줄임
8	쉴 때는 **[출력 끄기] (Space)**	서보가 손으로 돌아감 = 힘이 빠짐. 발열·전류 감소
9	끝낼 때 [연결 해제] 또는 창 닫기	둘 다 펄스 0 전송. 그 다음 서보 전원 끔

10-7. 안전 운용 수칙

- **전원**: 서보는 외부 5 V 2 A 이상. Pi 의 5 V 핀으로 돌리면 전압이 떨어져 Pi 가 재부팅되거나 SD 카드가 손상될 수 있습니다.
- **GND 공통**: 묶지 않으면 신호가 인식되지 않거나 서보가 떨립니다.
- **신호 3.3 V**: Pi GPIO 는 3.3 V 입니다. 대부분의 아날로그 서보가 인식하지만, 안 움직이면 레벨 변환기를 씁니다. 서보 쪽 5 V 를 GPIO 에 넣으면 안 됩니다.
- **한쪽에서만 조종**: Qt(Pi 화면) 와 WPF 가 동시에 접속하면 마지막 명령이 이깁니다. 헷갈리니 한쪽만 씁니다.
- **힘을 빼 두기**: 위치를 유지할 필요가 없으면 출력을 끕니다. 부하가 걸린 채 오래 유지하면 발열·전류가 큽니다.
- **끝단 소리**: '지잉' 이 계속 나면 즉시 반대쪽으로 움직이거나 출력을 끄고 범위를 줄입니다.

10-8. 에뮬레이터 운용 — Pi 가 없을 때

방법	절차
A. 앱 내장 에뮬레이터 (가장 쉬움)	연결 방식 **에뮬레이터** → [연결]. 상단에 "가짜 Pi 를 127.0.0.1 에 띄움", "Pi 리비전 c03111 · pigpio v79", 그리고 명령마다 "가짜 Pi 수신 #n: SERVO gpio=18 1700 µs". 그 뒤는 실제와 똑같이 조작.

B. 콘솔 에뮬레이터 `pigpio-emu`	`dotnet run --project PigpioEmulatorCli` (127.0.0.1:8888) 를 띄우고, WPF 를 **직접 연결 · 127.0.0.1 · 8888** 로 접속. 콘솔에 받은 명령과 서보 위치 막대가 찍힘:
	`09:33:48.016  ✓ SERVO gpio=18 1588 µs`
	`GPIO18 [──────────┼─────────] 현재`
	`+8.8°  1588 µs → 목표  +8.8°`

C. WSL 의 Qt 앱으로 시험	`dotnet run --project PigpioEmulatorCli -- 8888 0.0.0.0` 로 모든 주소에서 듣게 한 뒤, WSL 에서 Qt ServoControl 을 Windows 호스트 주소(WSL 에서 `ip route | grep default` 로 확인) 로 접속. Pi 없이 Qt 쪽도 검증 가능. (Windows 방화벽이 물으면 허용)
D. WSL 을 가짜 Pi 로 (리눅스 쪽 가짜 pigpiod)	WSL 터미널에서 `./run-from-wsl.sh fakepi` (= `python3 tools/fake_pigpiod.py`) → WPF 는 **직접 연결 · 127.0.0.1 · 8888**. 명령이 Windows 에서 WSL 로 실제 네트워크를 건너간다. WSL 에 sshd 를 넣으면 "WSL 터널" 모드까지 같은 서버로 시험 가능 (13-2 절)

에뮬레이터는 펄스 검사·리비전 응답·서보 속도(0.19 s/60°) 까지 흉내 내지만, 실제 부하·끝단·전원 문제는 재현하지 못합니다. "통신과 프로그램 논리" 를 검증하는 도구로 쓰고, 보정값은 실제 서보로 다시 맞추세요.

10-8-1. WSL 터미널만으로 운용하기 (요약)

자세한 원리와 확인 결과는 13-1·13-2 절. 실제 Pi 가 없어도 아래 순서로 전체 흐름을 연습할 수 있습니다.

1. WSL 터미널 1: `cd ~/MyJob/ServoChapter/ServoControlWpf && ./run-from-wsl.sh fakepi` — WSL 이 가짜 Pi 가 되어 8888 에서 기다림
2. WSL 터미널 2: `./run-from-wsl.sh` — Windows 화면에 WPF 창이 뜸
3. WPF: 연결 방식 **직접 연결**, Pi 주소 `127.0.0.1`, 포트 `8888` → [연결] → [0°] → 다이얼 조작. 터미널 1 에 명령이 한 줄씩 찍힘

4. 실제 Pi 가 생기면 Pi 주소만 `192.168.10.2` 로 바꾸면 끝 (Pi 쪽 `pi_setup.sh` 선행)

10-9. 문제 해결표

증상	원인	조치
[연결] 하면 "연결할 수 없습니다"	Pi 꺼짐 / 케이블 / pigpiod 꺼짐 / 허용 주소 아님	[Pi 상태 확인] → `pigpiod: active` 인지. 아니면 [pi_setup.sh ↗]. 또는 WSL 터널 모드로 전환
"WSL 안의 ssh 가 바로 종료"	키 인증 안 됨, 별명 없음, 호스트 키 미확인	WSL 터미널에서 `ssh rpi` 가 되게 만든 뒤 재시도(10-4)
"10초 안에 터널 포트가 열리지 않았습니다"	Pi 응답 없음, 이더넷 끊김	`ping 192.168.10.2`, 케이블·고정 IP 확인
연결은 됐는데 서보가 안 움직임	서보 전원 없음 / GND 안 묶임 / 신호선 핀 틀림 / 3.3 V 인식 실패	펄스폭 숫자가 바뀌면 프로그램은 정상. 배선(12번 핀, GND 6번), 외부 5 V, 레벨 변환 확인
서보가 떨린다	전원 부족, GND 불량, 끝단 걸림	2 A 이상 전원, GND 공통, 범위 줄이기
"펄스폭은 0 또는 500~2500 µs"	보정값이 pigpio 허용 밖	최소 ≥ 500, 최대 ≤ 2500 으로 수정
"에뮬레이터를 띄울 수 없습니다"	포트 8888 을 다른 프로그램(pigpio-emu 등) 이 사용	Pi 포트를 바꾸거나, 직접 연결 127.0.0.1 로 그 프로그램에 접속
닫았는데 서보가 힘을 주고 있음	비정상 종료로 0 을 못 보냄	다시 연결 후 [출력 끄기], 또는 Pi 에서 `pigs s 18 0`
WSL 탭 글자가 깨짐	UTF-16/UTF-8 혼동	앱은 `-l` 은 UTF-16, 나머지는 UTF-8 로 처리함. 스크립트 자체가 다른 인코딩이면 `iconv`
WSL 셸에서 `./ServoControlWpf.exe` 가 "Permission denied"	Windows 에서 복사한 파일에 실행 비트가 없음	`chmod +x` 또는 `./run-from-wsl.sh` 사용
WSL 터널: "ssh: Could not resolve hostname rpi"	WSL 의 `~/.ssh/config` 에 Host rpi 가 없음	10-4 절대로 별명·키 등록. Pi 가 없으면 13-2 의 WSL sshd 로 대신

11. 안전 장치 (소프트웨어)

상황	동작
범위를 넘는 각도 요청	최소~최대 각도로 잘라서 출력 (`ClampAngle`)
확장값 ±90° 적용	확인 창 (끝단 충돌·기어 손상 경고)
통신 끊김·응답 없음	1.5 초 시간 초과 → 자동 연결 해제 → 0.5 초 재접속해 펄스 0 시도 → 오류 창
pigpio 가 거부 (−2, −7, −41)	한글 메시지, 연결은 유지, 모델 상태 불변
보정 적용 실패	새 사양으로 먼저 써 보고 성공했을 때만 반영. 입력칸은 실제 값으로 되돌림
창 닫기 · 종료 · 처리되지 않은 예외	정상 종료 경로로 펄스 0 전송, 터널/에뮬레이터 프로세스 종료, 설정 저장
잘못된 저장값	공식값으로 복구 (GPIO 번호는 유지). 손상된 JSON 은 기본값으로 시작
WSL 작업이 멈춤	[취소] 로 프로세스 트리 종료

12. 테스트와 검증 결과

12-1. 자동 테스트 (xUnit 53개, 모두 통과)

파일	개수	검증 내용
`ServoSpecTests`	17	+20° → 1700 µs, −40° → 1100, 범위 밖 clamp(표 데이터 여러 건), 비대칭 보정, 역변환, 기본값이 공식 HS-311, `IsValid` 경계값 여러 건
`PigpioClientTests`	6	가짜 pigpiod 로 바이트 열 `08 00 00 00 12 00 00 00 dc 05 00 00 00 00 00 00` 확인, −7 → 한글 메시지·연결 유지, 무응답 → 시간 초과·LinkLost, 접속 거부 안내문, 드라이버 Open 이 HWVER/PIGPV 읽고 Dispose 가 0 전송
`ServoModelTests`	8	SetAngle 실패 시 상태 불변, 잘못된 사양 거부, 사양 적용 실패 시 롤백, 범위 좁히면 각도 잘라 재출력, GPIO 변경 시 release→open, 드라이버 교체 시 이전 드라이버 release·dispose
`EmulatorTests`	3	Model → Driver → TCP → 에뮬레이터 전체 왕복(1700, 1100, 0 수신), 에뮬레이터의 −7/−2 거부, 흉내 낸 서보가 0.19 s/60° 로 목표에 도달
`ServoDialTests`	9	다이얼의 마우스 방향 → 각도(위 = 0°, ±135° = 끝, 45° → 13°, 빈 구간은 가까운 끝), 범위 변경 반영, 1° 단위 맞춤·클램프
`MainViewModelTests`	5	각도를 넣으면 펄스가(20° → 1700), 펄스를 넣으면 각도가(1300 → −20°) 바뀌고 드라이버로 출력, 범위 밖 펄스 클램프, 비대칭 보정 왕복(1655 ↔ 20.67°), 출력 실패 시 되돌림 + 오류 창
`ServoCameraViewTests`	4	카메라 뷰의 구동 조건: 연결 해제 상태에서 Angle 을 바꿔도 회전 타이머가 돌지 않고 그려진 각도가 제자리 / 연결 + 출력 켜짐이면 목표를 향해 회전 시작 / 연결됐어도 출력이 꺼져 있으면 정지, 출력이 켜지는 순간 출발 / 회전 중 연결을 끊으면 그 자리에서 정지
`WslSshTunnelTests`	1	없는 ssh 별명으로 터널을 열면 멈추지 않고 1초 안에 `ServoException`(ssh 의 오류 문장 포함)으로 실패 — WSL 이 있는 PC 에서 실제 `wsl.exe -e ssh` 를 실행해 확인

12-2. 화면 자동 조작 검증 (UI Automation, 이 PC · WSL 없음)

조작	결과
시뮬레이션에서 +10°, +1°, [현재 위치 저장]	"펄스폭 1610 µs", "+11.0°", P1 에 +11.0° 저장, 상태 바 "P1에 11.0° 저장"
스윕 시작 → 정지 → [출력 끄기]	"출력 꺼짐 · 손으로 돌릴 수 있음", "펄스폭 — µs"
탭 WSL → [WSL 확인]	로그: "이 PC에 WSL이 설치되어 있지 않거나 배포판이 없습니다. … 'wsl.exe --install'" (앱 정상 유지)
직접 연결 192.168.10.2 (Pi 없음) → [연결]	1.5 초 뒤 오류 창 "연결할 수 없습니다 … pi_setup.sh …", 버튼 "연결" 로 복구
에뮬레이터 → [연결] → +10° ×2 → 스윕 → 출력 끄기 → 연결 해제	"Pi 리비전 c03111 · pigpio v79", "가짜 Pi 수신 #4: SERVO gpio=18 1700 µs", 스윕 중 #72 … 1288 µs, 끄기 #138 "출력 끔", 해제 후 "연결 안 됨 (시뮬레이션)"
콘솔 pigpio-emu(18889) + 직접 연결 127.0.0.1:18889 → −10° → [P1]	"pigpio @ 127.0.0.1:18889 · GPIO18", 펄스폭 1610 µs, 콘솔에 "✓ SERVO gpio=18 1588 µs" 와 위치 막대
다이얼: −45° 방향 누름 → 0°/45°/90° 로 끌기 → 놓기 → 빈 구간(160°) 누름	−13°(1370 µs) → 1500/1630/1770 µs → 다이얼 27 → 가까운 끝 40°(1900 µs). [+10°] 버튼을 누르면 다이얼이 따라 돎
각도 ↔ 펄스 (보정 800/1500/2100): 펄스 슬라이더 1650 / 다이얼 30 / 펄스 입력칸 1234 / 각도 입력칸 −5 / 펄스 2400	각도 10.0° / 펄스 1950 / 각도 −15.2° / 펄스 1413 / 최대 2100 으로 잘려 40.0°. 매번 가짜 Pi 가 같은 µs 를 수신
카메라 뷰: −40° → +40° 명령 60 ms 뒤 캡처 / 왼쪽 화면 클릭	"+3.4° → 목표 +40.0° 로 회전 중" 표시 후 0.25 s 안에 +40° 도달 / 클릭 전후 각도 10 → 10 (입력을 받지 않음)
카메라 뷰 연결 조건: 연결 전 다이얼 30° → 에뮬레이터 연결 → −20° → 연결 해제 → 다이얼 40° → 재연결 → 39°	연결 전·해제 후에는 "연결 안 됨" 안내만 보이고 카메라 제자리 (0° → 연결 후 −20° 도달 → 해제 뒤 40° 로 돌려도 −20° 유지). 재연결 직후 회색 카메라 −20° "출력 꺼짐", 첫 명령 39° 에 다시 회전 (그림 6-1)
[종료]	프로세스 종료, `ServoControl.json` 에 모드·주소·보정·P1 저장됨



그림 7. 처음 실행한 화면 (연결 전, 출력 꺼짐). 왼쪽 카메라가 회색이고 시야가 없다 = 힘이 빠진 상태. 이 PC 에서 캡처.

12-3. 검증 중 찾아서 고친 문제

증상	원인	수정
WSL 터널 모드에서 [연결] 을 누르면 ssh 가 바로 실패하는데도 오류 창이 뜨지 않고 "연결 중..." 상태로 남음 (WSL 을 설치한 뒤 처음 드러남)	`WslSshTunnel.OpenAsync` 가 ssh 프로세스를 정리(Dispose)한 **뒤에** 종료 코드를 읽어 다른 종류의 예외가 나고, 그 예외는 ViewModel 이 잡지 않았음	종료 코드를 먼저 읽고 정리하도록 순서 변경. ViewModel 은 예상 못 한 예외도 잡아 "연결 중 예상하지 못한 오류" 창으로 보여줌. 재발 방지 테스트 `WslSshTunnelTests` 추가(ssh 별명이 없으면 1초 안에 안내문과 함께 실패해야 함)
WSL 리눅스 파일시스템에 복사한 .exe 가 "Permission denied"	Windows 에서 복사하면 실행 비트가 없음	`run-from-wsl.sh` 가 `chmod +x` 후 실행

12-4. 실제 PC(WSL + Pi) 에서 할 확인 목록

1. 직접 연결 → [연결] → "Pi 리비전 ... · pigpio v..." → [0°] 서보 중립 → 스윕 → [출력 끄기] 로 힘 빠짐
2. WSL 터널 → [연결] → 동일 동작. Pi 에서 `ss -tn | grep 8888` 에 127.0.0.1 접속이 보임
3. WSL 탭 → [Pi 상태 확인] 출력, [pi_info.sh] 결과(pi_info.txt) 표시, [pi_setup.sh ↗] 새 터미널 창
4. 창 닫기 후 서보가 손으로 돌아가는지 (펄스 0 전송 확인)

13. 만들기·실행하기

```
# 필요: .NET SDK 8 이상 (SDK 10 으로 빌드 확인), Windows 10/11
cd ServoChapter\ServoControlWpf
dotnet build ServoControlWpf.sln -c Release      # 경고 0, 오류 0
dotnet test  ServoControlWpf.sln -c Release      # 34 통과
dotnet run   --project ServoControlWpf           # 앱 실행

# 가짜 pigpiod 콘솔
dotnet run --project PigpioEmulatorCli                   # 127.0.0.1:8888
dotnet run --project PigpioEmulatorCli -- 8888 0.0.0.0   # 모든 주소 (WSL 의 Qt 앱용)
```

항목	값
실행 파일	`ServoControlWpf\bin\Release\net8.0-windows\ServoControlWpf.exe`
설정 파일	`%APPDATA%\VI\ServoControl.json` — 주소·포트·모드·배포판·ssh 별명·터널 포트·스크립트 폴더·보정값·P1~P4·스윕 속도
기본값	192.168.10.2:8888, 직접 연결, rpi, 18888, 보정 1100/1500/1900 µs ±40° GPIO18
스크립트 폴더	처음 실행 시 실행 파일 위쪽에서 `ServoControl\pi_setup.sh` 를 찾아 자동 설정. WSL 내부 경로(`/home/…`) 도 됨
WSL 준비 (실제 PC)	`wsl --install -d Ubuntu` → WSL 에서 `ssh-keygen`, `ssh-copy-id user@192.168.10.2`, `~/.ssh/config` 에 `Host rpi`
이 문서 만들기	`ServoControlWpf_설계과정.html` 을 Edge 로 PDF 인쇄: `msedge --headless --print-to-pdf=… file:///…html`

13-1. WSL 셸에서 실행하기 — "통신만으로" 띄우고 연결하기

프로그램 폴더를 WSL 의 리눅스 파일시스템 (`\\wsl.localhost\Ubuntu\home\jhjung\MyJob\ServoChapter\ServoControlWpf`)에 두고, **WSL 터미널에서 명령만으로** 실행하고 연결할 수 있는지 이 PC 에서 확인했습니다 (실제 Pi 는 연결되어 있지 않음).

> **원리 — WPF 는 리눅스에서 돌지 않지만, WSL 은 Windows 프로그램을 "대신 띄워" 준다**
>
> WSL 의 **interop** 기능 덕분에 리눅스 셸에서 `.exe` 를 실행하면 Windows 가 그 프로그램을 띄웁니다. 창은 Windows 화면에 뜨고, 네트워크·설정 파일(`%APPDATA%`)도 Windows 쪽을 씁니다. 즉 "WSL 에서 실행"은 **Windows 프로그램을 WSL 터미널에서 원격 조종하듯 띄우는 것**이고, 그 뒤의 연결(직접·터널·에뮬레이터)은 Windows 에서 띄웠을 때와 완전히 같습니다. 리눅스용 .NET 을 설치해도 WPF 는 실행되지 않습니다.

WSL 셸에서 띄우는 방법	결과	비고
A. `./ServoControlWpf.exe` 직접 실행 (리눅스 FS 의 .exe)	✓	Windows 에서 복사해 넣은 파일은 실행 비트가 없어 "Permission denied" → `chmod +x` 한 번 필요. Windows 는 이 파일을 `\\wsl.localhost\…` 경로로 읽어 실행
B. `dotnet.exe "$(wslpath -w …/ServoControlWpf.dll)"`	✓	실행 비트 불필요. 경로는 `wslpath -w` 로 Windows 형식(UNC)으로 바꿔 줘야 함 (리눅스 경로를 주면 "예상 원인" 오류)
C. `powershell.exe Start-Process`	△	WSL 셸이 돌아오지 않고 멈추는 경우가 있어 권장하지 않음
D. `cmd.exe /c start "" "\\wsl.localhost\…"`	✗	cmd 는 UNC 경로를 지원하지 않음
E. 리눅스 `dotnet run --project ServoControlWpf`	✗	WPF 는 Windows 전용 (net8.0-windows). 리눅스 .NET 은 `NETSDK1100` ("Windows 대상 프로젝트는 이 OS 에서 빌드할 수 없다") 오류를 냄. `Directory.Build.targets` 가 이 상황을 미리 감지해 "./run-from-wsl.sh build 를 쓰세요" 라는 한글 안내로 바꿔 줌
F. 리눅스 `dotnet run --project PigpioEmulatorCli -- 8888 0.0.0.0`	✓	가짜 pigpiod(`pigpio-emu`)는 WPF 를 쓰지 않는 보통 콘솔 프로그램이라 리눅스 .NET 10 에서도 돌아감(RollForward). D 의 python 판 `fakepi` 와 같은 역할

리눅스 dotnet 설치 중 겪은 문제와 결론

WSL 에 `snap install dotnet` 으로 넣은 .NET 10 은 `libunwind8` 라이브러리가 없어 `dotnet --version` 부터 실패했고, Ubuntu 26.04 의 apt 저장소에는 `dotnet-sdk-8.0` 이 없어 `dotnet-sdk-10.0` 을 설치해야 했습니다. 설치가 끝난 뒤에도 WPF 는 위 E 처럼 빌드되지 않습니다. **결론: 리눅스 dotnet 은 이 프로그램을 쓰는 데 필요하지 않습니다.** WPF 실행은 항상 Windows 의 `dotnet.exe` / `.exe` 가 맡고, `run-from-wsl.sh` 가 그 일을 대신합니다. 리눅스 dotnet 이 이미 있다면 F 처럼 가짜 pigpiod 를 리눅스 프로세스로 띄우는 데에만 쓸 수 있습니다.

A 방법을 스크립트로 묶은 `run-from-wsl.sh` 를 프로젝트 폴더에 두었습니다.

```
# WSL 터미널에서
cd ~/MyJob/ServoChapter/ServoControlWpf
chmod +x run-from-wsl.sh
./run-from-wsl.sh            # Release(없으면 Debug) 실행 파일을 띄움. 터미널은 바로 돌아옴
./run-from-wsl.sh build      # Windows 의 dotnet.exe 로 먼저 빌드한 뒤 띄움
./run-from-wsl.sh emu 8888   # Windows 쪽 가짜 pigpiod 콘솔을 이 터미널에서 실행
./run-from-wsl.sh fakepi     # WSL 자체를 가짜 Pi 로: 리눅스 프로세스 tools/fake_pigpiod.py
(0.0.0.0:8888)
# 리눅스 dotnet 이 설치돼 있다면 .NET 판 가짜 Pi 도 가능:
dotnet run --project PigpioEmulatorCli -- 8888 0.0.0.0
```

WSL 에서 띄운 앱으로 확인한 것	결과
Windows 가 보는 실행 경로	`\\wsl.localhost\Ubuntu\home\jhjung\MyJob\ServoChapter\ServoControlWpf\…\ServoControlWpf.exe`
탭 WSL → [WSL 확인]	"WSL 사용 가능 · 배포판 1개", 로그 "배포판: Ubuntu"
연결 방식 WSL 터널 → [연결] (ssh 별명 `rpi` 가 없는 상태)	1초 안에 오류 창: "WSL 안의 ssh 가 바로 종료되었습니다 (종료 코드 255). ssh: Could not resolve hostname rpi …" + 점검 안내
연결 방식 에뮬레이터 → [연결] → 다이얼 20	"가짜 Pi 수신 #3: SERVO gpio=18 1700 µs" — WSL 에서 띄운 앱도 TCP 명령 경로가 그대로 동작
스크립트 폴더를 WSL 쪽 `\\wsl.localhost\…\ServoControl` 로 두고 [Pi 상태 확인]	`wsl --cd` 가 UNC 경로도 받아들임. 로그: "ssh: Could not resolve hostname rpi" (Pi 없음 — 정상 오류)
[종료]	정상 종료, WSL 셸에 남는 프로세스 없음
`./run-from-wsl.sh build` (WSL 셸에서 Windows 의 dotnet.exe 로 UNC 경로 빌드)	"빌드했습니다. 경고 0개, 오류 0개" (약 5.5 초) 후 Release 실행 파일이 뜸
`./run-from-wsl.sh fakepi` + Windows WPF 직접 연결 127.0.0.1:8888	WSL 의 리눅스 가짜 pigpiod 가 HWVER·PIGPV·SERVO 를 수신 (13-2 로그)
리눅스 dotnet 10 으로 `pigpio-emu 8888 0.0.0.0` 실행 + Windows 에서 127.0.0.1:8888 접속	리눅스 쪽 로그 "✓ PIGPV → 79", "✓ SERVO gpio=18 1700 µs", 막대가 +20.0° 로 이동 — .NET 판 가짜 Pi 도 WSL 안에서 동작
리눅스 dotnet 으로 `dotnet run --project ServoControlWpf`	NETSDK1100 대신 한글 안내 5줄("WPF 는 Windows 에서만 … ./run-from-wsl.sh build 를 사용하세요") 후 중단. `PigpioEmulatorCli` 빌드는 영향 없음(0 오류)

13-2. Pi 없이 "WSL 을 가짜 Pi" 로 만들기 (두 단계)

에뮬레이터 모드는 앱 **안**에서 가짜 Pi 를 띄우지만, 네트워크를 건너는 모습까지 보고 싶으면 WSL 에 가짜 pigpiod 를 띄웁니다.

1. **직접 연결로 시험** (sudo 불필요, **이 PC 에서 확인됨**): WSL 에서 `./run-from-wsl.sh fakepi` → 리눅스 프로세스가 0.0.0.0:8888 에서 듣는다. WSL2 는 WSL 안의 포트를 Windows 의 `127.0.0.1` 로도 열어 주므로 (localhost 포워딩), WPF 에서 **직접 연결 · Pi 주소 127.0.0.1 · 포트 8888** → [연결]. 이때 명령은 Windows → (가상 네트워크) → WSL 리눅스 프로세스로 실제로 건너간다. Qt ServoControl 을 WSL 안에서 실행해 127.0.0.1:8888 로 붙여도 된다.

```
# WSL 쪽 가짜 pigpiod 가 찍은 로그 (WPF 를 직접 연결 127.0.0.1:8888 로 붙인 뒤 다이얼 -20° → +30° → 출력 끄기)
● 접속 127.0.0.1:60440
11:14:35  ✓ HWVER → 0xc03111
11:14:35  ✓ PIGPV → 79
11:14:40  ✓ SERVO gpio=18 1300 µs → 목표 -20.0°
11:14:43  ✓ SERVO gpio=18 1800 µs → 목표 +30.0°
11:14:45  ✓ SERVO gpio=18 출력 끔
○ 해제 127.0.0.1:60440
```

WPF 쪽 연결 바에는 "pigpio @ 127.0.0.1:8888 · GPIO18 · Pi 리비전 c03111 · pigpio v79" 가 떴다. 즉 **Pi 대신 WSL 이 응답해도 프로그램은 구별하지 못한다**.

2. **WSL 터널로 시험** (sudo 필요, 한 번만): WSL 에 SSH 서버를 넣고 자기 자신을 `rpi` 라는 별명으로 등록하면, 앱의 "WSL 터널" 모드가 WSL 의 sshd 를 거쳐 1번의 가짜 pigpiod 에 닿는다.

```
sudo apt install -y openssh-server && sudo systemctl enable --now ssh
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519      # 키가 없을 때
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys
printf 'Host rpi\n    HostName 127.0.0.1\n    User %s\n' "$USER" >> ~/.ssh/config
ssh -o StrictHostKeyChecking=accept-new rpi true && echo "ssh rpi OK"
```

그 다음 WPF 에서 **WSL 터널 · ssh 별명 rpi · Pi 포트 8888** → [연결]. 이 PC 에서는 sudo 비밀번호가 필요해 2번은 실행하지 못했고, 사용자가 위 명령을 직접 실행한 뒤 확인해야 합니다.

14. 한계와 다음 단계

- **실제 Pi 검증은 아직**: 실제 장치가 물리적으로 연결되어 있지 않아 직접·터널 모드는 에뮬레이터, 가짜 pigpiod, 오류 경로로만 확인했습니다. 이 PC 에 WSL(Ubuntu)은 설치되어 WSL 셸 실행·WSL 탭·터널의 실패 경로까지는 확인했지만(13-1), 터널의 성공 경로는 SSH 서버(Pi 또는 13-2 의 WSL sshd)가 있어야 합니다. 12-4 목록으로 실제 PC 에서 확인이 필요합니다.
- **WSL 터널의 비밀번호 입력 불가**: 설계상 키 인증 전제. 비밀번호 방식이 꼭 필요하면 터널도 "새 터미널 창" 으로 여는 선택지를 추가할 수 있습니다.
- **서보 명령은 동기 호출**: Qt 와 같이 1.5 초 타임아웃 동안 화면이 잠깐 멈출 수 있습니다(연결 단계만 비동기). 사용감이 문제되면 명령 큐 + 백그라운드 스레드로 바꿀 수 있습니다.
- **다음 단계 후보**: 여러 서보(채널) 동시 제어, 동작 시퀀스 녹화·재생, 에뮬레이터에 부하·끝단 흉내, mDNS 로 Pi 자동 찾기.

14-1. 이 프로그램을 Pi 로 옮길 수 있나 — 조사 결과

결론: 그대로는 안 되고, 화면 층만 바꾸면 됩니다. WPF 는 Windows 전용 프레임워크라 Raspberry Pi OS(리눅스)에서는 실행되지 않습니다. 하지만 이 프로그램은 5장처럼 층이 나뉘어 있어서, **서보·통신·설정을 아는 Model/Services 층은 손대지 않고** 화면(View)과 ViewModel 의 타이머 두 줄만 바꾸면 Pi 에서 돌릴 수 있습니다. 이 PC 에서 확인한 사실은 다음과 같습니다.

확인한 것	결과
Model 9개 · Services 2개 · ViewModel 밑바탕 4개 파일을 WPF 없이 리눅스 .NET 10 으로 컴파일 (`tools/portcheck`, WSL 에서 `dotnet build tools/portcheck`)	**경고 0 · 오류 0.** 이 층들은 `System.Windows` 를 전혀 쓰지 않는다
이 PC 에서 Pi 용 실행 파일 만들기 (`dotnet publish PigpioEmulatorCli -r linux-arm64 --self-contained`)	단일 파일 `pigpio-emu` (70 MB, ELF 64-bit AArch64) 생성. Pi 4 의 64-bit OS 에서 .NET 설치 없이 실행 가능한 형식
WPF 에 묶인 파일	`View\` 5개(MainWindow, ServoDial, ServoCameraView, DialogService, App)와 `MainViewModel` 의 `DispatcherTimer` 2개(스윕·상태 줄). `RelayCommand` 의 `ICommand` 는 리눅스에서도 있는 기본 형식이라 문제없음
Pi 에서 의미가 없어지는 부분	WSL 터널 모드와 "WSL / Pi 관리" 탭(Pi 안에서는 WSL 이 없음). Pi 에서는 **직접 연결 · 주소 127.0.0.1 · 포트 8888** 하나면 충분 — pigpiod 가 같은 기기 안에 있음

세 가지 길 비교

방법	내용	판단
A. Avalonia 로 화면만 이식 (권장)	WPF 와 거의 같은 XAML·바인딩·`DrawingContext` 를 가진 크로스플랫폼 UI 프레임워크. Model/Services 는 그대로 두고 `View\` 5개 파일과 타이머 2줄을 Avalonia 판으로 다시 씀. `dotnet publish -r linux-arm64` 로 Pi 용 단일 실행 파일을 만들어 `deploy.sh` 처럼 ssh 로 복사·실행	작업량: 다이얼·카메라 뷰 그리기 코드 이식이 대부분(각 200~270줄). 2~3일 규모. 다이얼 UI 자동화 피어는 Avalonia 식으로 다시 작성
B. 기존 Qt 판 사용	Pi 에서 돌 목적으로 이미 만들어진 `ServoControl` (Qt/C++)이 `deploy.sh` 로 Pi 에서 빌드·실행됨	지금 당장 Pi 화면에서 조종하려면 이것이 가장 빠름. 단, 카메라 뷰·펄스 입력 같은 WPF 판의 새 기능은 없음
C. Pi 에 Windows 설치	Pi 4 에 Windows on ARM 을 올리는 비공식 방법(WoR). WPF 가 그대로 돌긴 함	공식 지원 없음, 느림, GPIO/pigpiod 가 리눅스용이라 서보 제어 자체가 막힘. **권하지 않음**

A 방식의 작업 순서 (Pi 4, Raspberry Pi OS 64-bit 기준)

1. 프로젝트 `ServoControlAvalonia` 를 만들고 `Model\`·`Services\`·`ViewModel\` (ObservableObject, RelayCommand, IDialogService, PresetSlot, MainViewModel) 파일을 링크로 포함한다 (`tools/portcheck/PortCheck.csproj` 의 `<Compile Include="..\ServoControlWpf\Model\*.cs"/>` 방식). 소스 복사본이 아니라 **같은 파일**을 두 앱이 쓴다.
2. `MainViewModel` 의 `DispatcherTimer` 를 Avalonia 의 `DispatcherTimer` (`Avalonia.Threading`)로 바꾼다 — 이름과 쓰임이 같아 using 한 줄 차이. WSL 관련(`Wsl` 속성, 터널 모드)은 `#if` 나 별도 ViewModel 로 떼어 Pi 판에서는 숨긴다.

3. `ServoDial`·`ServoCameraView` 를 Avalonia `Control` 로 이식한다. `OnRender(DrawingContext)` → `Render(DrawingContext)`, `DependencyProperty` → `StyledProperty`, `FormattedText` 생성자 인자만 다르고 그리기 호출(`DrawGeometry`, `DrawEllipse`, `PushTransform`)은 거의 같다.
4. `MainWindow.axaml` 은 WPF XAML 을 거의 그대로 옮긴다(`Visibility` → `IsVisible`, `GroupBox` → `HeaderedContentControl` 정도).
5. PC 에서 `dotnet publish ServoControlAvalonia -c Release -r linux-arm64 --self-contained -p:PublishSingleFile=true` → `scp` 로 Pi 에 복사 → Pi 에서 `chmod +x` 후 실행. 접속은 **직접 연결 127.0.0.1:8888**. Pi 쪽 `pi_setup.sh` 는 이미 pigpiod 를 켜 두므로 추가 설정 없음.
6. 검증은 Pi 없이도 가능: WSL 에서 `-r linux-x64` 로 publish 해 WSLg 창으로 띄우고(Avalonia 는 WSLg 에서 동작) `./run-from-wsl.sh fakepi` 의 가짜 pigpiod 에 127.0.0.1:8888 로 붙여 본다.

왜 Avalonia 인가

.NET MAUI 는 리눅스를 지원하지 않고, Uno Platform 은 가능하지만 WPF 와의 거리가 더 멉니다. Avalonia 는 "리눅스에서 도는 WPF" 에 가장 가까워, 이 프로젝트처럼 `OnRender` 로 직접 그린 컨트롤과 MVVM 바인딩을 가장 적게 고치고 옮길 수 있습니다. Pi 4 에서 Avalonia 앱은 X11 데스크톱(기본 Raspberry Pi OS) 위에서 그대로 뜨며, 화면 없이 직접 프레임버퍼로 띄우는 방법(DRM)도 있습니다.

참고 — Raspberry Pi 4 Model B Datasheet Rel. 1.1 · HS-311 사양서 · pigpio Socket Interface (abyz.me.uk/rpi/pigpio/sif.html) · WSL 네트워킹 문서(Microsoft Learn) · 본 프로젝트의 ServoControl 동작 원리 PDF. 확인일 2026. 10. 26.